Agentic Customer Support Assistant

Senior AI Engineer (Text + Voice) — Technical Assignment

Duration	3 Days (72 hours from receipt)
Deliverable	GitHub Repository + Presentation (PPT) + Design Notes + Part C Answers + Voice demo recordings
Submission	Share the GitHub link + design notes, demo and slides to [submission email/drive link] before the deadline
Prerequisite	None — build your own small knowledge base and source your own voice audio
Dataset	None provided — write your own 10–20 short knowledge-base docs (FAQs, shipping/returns, troubleshooting) anchored to a product/service of your choosing
Tech Stack	No fixed language or framework — use whatever you're comfortable with. Groq's API for LLM completions and Whisper-based STT (free tier). Any free/open TTS (Edge TTS, gTTS, pyttsx3, etc.). Embedding approach for retrieval is your choice — justify it, since Groq doesn't serve embeddings.

Scenario

You're building a customer support assistant that works over both text and voice. On the text side it needs to route incoming messages to the right handler, ground its answers in a real knowledge base instead of hallucinating, call tools for things like order status, and refuse to be talked into unauthorized actions. On the voice side it needs to behave like an actual phone line: it has to know when it's being interrupted, recover cleanly when a provider fails mid-call, and respond fast enough that the latency doesn't break the illusion of a conversation.

Setup is part of the assessment, not a precondition for it. Source or write your own small knowledge base (10–20 short docs: product FAQs, shipping/returns policy, troubleshooting guides) anchored to a product or service, real or invented. Choose and justify your own embedding approach for retrieval, since Groq doesn't serve embeddings. For the voice portion you'll need real audio — your own recorded clips are fine. If you hit free-tier rate limits on Groq, creating additional accounts/keys is fine and expected.

Detailed Tasks & Deliverables

Part A — Agentic Support System

Deliverable: multi-agent router built on Groq as the LLM backend

- **Router** — classify incoming messages into ≥3 intents (e.g. order_status, product_question, escalate_to_human) and dispatch each to a distinct agent/handler.

- **RAG agent** — one path retrieves from your knowledge base via embeddings and grounds its answer in retrieved passages (not keyword search). This handles product_question.
- **Tool-calling agent** — one path calls a function you define (e.g. get_order_status(order_id), check_return_eligibility(order_id)) using Groq's function-calling support or your own ReAct-style loop — justify the choice.
- **Guardrail** — one input-side check (prompt injection — e.g. a user trying to get the bot to issue an unauthorized refund or reveal internal instructions) and one output-side check (a forbidden-claim category you define, e.g. "never promise a refund amount or delivery date the system hasn't confirmed"). Then trace your own router/guardrail logic end to end and show structurally — not by assuming it — that there is no path by which an unchecked message reaches the RAG agent or the tool-calling agent.
- **Cache** — cache repeated classifier calls against Groq, report hit rate.

Note: the output-side guardrail ("never promise an unconfirmed refund amount") and the tool-calling agent (which can resolve a return/refund eligibility check in one turn) can pull in opposite directions. Don't let one silently override the other — document explicitly how your design reconciles the two.

Design note (required): How does this scale to 10k concurrent sessions? How do you add a 4th agent (e.g. billing_dispute) without breaking existing routing? What's your eval set and metrics for routing accuracy and guardrail recall?

Part B — Voice Turn-Taking Pipeline

Deliverable: a real pipeline — your mic/audio input → Groq Whisper STT → your chosen TTS for output — framed as a phone-support line.

- **Barge-in:** while the assistant is "speaking" (e.g. reading out order details), detect real incoming user speech vs. background noise, cancel playback cleanly, and hand off to STT without double-handling a turn. Demonstrate 3 scenarios: silence through full playback, a short noise burst that should not interrupt, and a real utterance that should (e.g. the user interrupting with "wait, wrong order"). Add at least one adversarial case of your own choosing (e.g. overlapping speech, a false-start then real interruption) and show how your coordinator resolves it.
- **Provider resilience:** simulate your TTS or STT provider failing mid-call (kill network, invalidate key) and show fallback/retry behavior — no hangs or crashed turns.
- **Latency instrumentation:** measure and report end-to-end latency — time from end-of-user-speech to first audio chunk of the response.

Submit a short screen-recorded demo or clear logs/transcripts showing each scenario running.

Design note (required): How would you detect end-of-utterance robustly across accents/languages with different pause patterns? On-device vs. server-side VAD — tradeoff? Streaming vs. batch ASR — when would you pick each?

Part C — Integration & Ops

Deliverable: written answers, no code

- A user starts a call in voice and is transferred to text chat mid-conversation (e.g. moving to a chat transcript for a human agent) — how do Part A and Part B share state?
- What's your production eval strategy (safety/guardrail regression, latency SLOs, A/B testing a routing change) for this combined system?

- Groq degrades mid-launch (high latency, partial outage) during a high-volume support day. What's your detection + fallback plan — would you failover to a different LLM provider, and what does that cost you in behavior consistency?

Process & Integrity Requirements

This assignment is scored on evidence of genuine engineering judgment, not on whether the final code runs. Using AI tools to help you write code is fine — the requirements below exist specifically so that a submission produced without real understanding of the system doesn't pass, regardless of how it was typed.

- **Real commit history:** Submit your actual git history, not a single squashed commit. Incremental commits over the work period are expected; a one-shot dump is treated as a red flag, not a shortcut.
- **Decisions log:** Keep a short, timestamped log of design decisions as you made them — including approaches you tried and abandoned and why. This is evaluated separately from the design notes; a polished retrospective with no dead ends is unconvincing.
- **Red-team your own system:** Find and document at least two cases where your own guardrail, router, or barge-in logic fails, and explain honestly why you didn't fix them (time, approach limitation, etc.). An audit with zero acknowledged weaknesses across every component will be treated with suspicion, not credit.
- **Unique scenario parameters:** Your knowledge base, product domain, and at least one adversarial test case must be your own and specific to your submission — not a generic template. Reviewers will ask you to justify choices that are specific to your data.
- **Live defense round:** Shortlisted candidates will be asked to extend their own system live with a new constraint given on the spot.

Constraints & Rules

- **API Keys:** Don't commit or share your real Groq API key anywhere, including in the write-up or demo recording.
- **Resourcefulness:** Free-tier limits are expected, not an excuse — creating additional Groq accounts/keys to work around rate limits is fine, but handle the constraint sensibly rather than letting it block the submission.
- **Real audio:** The voice pipeline must be exercised with real recorded audio clips, not synthetic-only or mocked input — barge-in and latency claims should be demonstrable against actual speech.
- **Grounding, not guessing:** The RAG agent must ground its answers in retrieved passages; it must not hallucinate order details, policy terms, or other specifics it hasn't actually retrieved or been given via a tool call.
- **Guardrails must hold under pressure:** Input- and output-side checks need to catch real support-bot failure modes (unauthorized refunds, policy overpromising) — superficial keyword/token matching is not sufficient.
- **No dropped or duplicated turns:** The voice coordinator must avoid race conditions between playback, barge-in detection, and STT handoff; a provider failure must degrade gracefully (fallback/retry), never hang or crash a turn.

- **Explain your numbers:** Reported latency figures must be instrumented, not estimated, and you should be able to explain why the number is what it is.
- **Reasoned design notes:** The required design notes and Part C answers must reason about scale, evaluation, and failure modes — not simply describe what the code does.
- **Reproducibility:** Your submission must contain all code, both design notes, the Part C answers, and the voice demo recording or logs, organized so a reviewer can follow what ran and what it produced.
- **Honesty over polish:** If you're unsure whether something is genuinely solved or just looks solved, say so explicitly in the decisions log or design notes — that is scored as a positive, not a weakness.